

Mini-projet 11 — Les générateurs (yield)

Module C · puissance Notions : **yield** · **évaluation paresseuse** · **itérateur à usage unique** · **expression génératrice** · Durée : 1 h 30
· Parcours Python — Ednir

Pourquoi ce mini-projet ?

Un scanner doit balayer des **plages énormes** : les 65 535 ports, ou toutes les IP d'un sous-réseau. Si tu construis d'abord une **liste** de tout ça, tu remplis la RAM pour rien. Un **générateur** produit les valeurs **une par une, à la demande** — il n'en garde jamais plus d'une à la fois. C'était un de tes gros trous du test de départ ; on le règle aujourd'hui.

1. Le problème que ça résout

```
# Version LISTE : construit et garde les 10 millions de nombres en RAM d'un coup
def nombres():
    return [x for x in range(10_000_000)]    # ~400 Mo de mémoire !
```

Tu n'as peut-être besoin que du 1er nombre, ou de t'arrêter dès que tu trouves un port ouvert. Pourtant la liste construit **tout**. Le générateur, lui, ne produit que ce qu'on lui demande.

2. yield : produire une valeur et se mettre en pause

Une fonction qui contient **yield** (au lieu de **return**) devient un **générateur**. À chaque **yield**, elle **rend une valeur puis se met en pause** — elle reprendra exactement là où elle s'était arrêtée au tour suivant.

```
def compte():
    yield 1      # rend 1, PAUSE ici
    yield 2      # au tour suivant, reprend, rend 2, PAUSE
    yield 3      # rend 3, puis c'est fini

for n in compte():
    print(n)      # 1, puis 2, puis 3
```

L'image : **return** = « je réponds et je pars pour de bon ». **yield** = « voici une valeur, je me mets en pause, réveille-moi pour la suivante ». La fonction **garde sa place** entre deux appels.

La version « plage » avec une boucle

```
def plage_ports(debut, fin):
    port = debut
    while port <= fin:
        yield port      # rend UN port, se met en pause
        port += 1        # reprend ici au tour suivant

for p in plage_ports(20, 23):
    print(p)            # 20, 21, 22, 23 — jamais plus d'un en mémoire
```

3. Paresseux : rien n'est calculé tant qu'on ne demande pas

```
g = plage_ports(1, 65535)    # INSTANTANÉ : rien n'est encore calculé
print(g)                    # <generator object ...> — un "robinet", pas les valeurs
print(next(g))               # 1 — on demande la 1re valeur, elle sort
print(next(g))               # 2 — la suivante
```

- **next(g)** demande la **prochaine** valeur. C'est ce que fait **for** en coulisses.
- Un générateur est un **robinet** : il coule goutte à goutte, quand tu ouvres.

Comparaison mémoire (frappant)

```
import sys
liste = [x for x in range(1000000)]    # crochets [] = une vraie liste
gene = (x for x in range(1000000))     # parenthèses () = un générateur
print(sys.getsizeof(liste))            # ~8 000 000 octets (8 Mo)
```

```
print(sys.getsizeof(gene)) # ~200 octets (!) — il ne stocke rien, il calcule à la demande
```

Expression génératrice : comme une compréhension de liste (MP01) mais avec des **parenthèses** au lieu des crochets. [...] construit tout ; (...) produit à la demande.

4. ⚠ Le piège : un générateur ne se parcourt QU'UNE fois

Une fois épuisé (toutes les valeurs sorties), il est **vide**. Il ne se « rembobine » pas.

```
g = plage_ports(20, 22)
list(g) # [20, 21, 22] — on l'a tout consommé
list(g) # [] — VIDE ! il est épuisé
```

Conséquence : si tu as besoin de parcourir les valeurs **deux fois**, soit tu recrées le générateur (rappelle la fonction), soit tu le convertis une fois en liste avec `list(g)` (mais tu perds l'économie de mémoire). Choisis selon ton besoin.

🎯 Ta mission

Objectif : écrire deux générateurs qui produisent des plages sans les stocker.

- `plage_ports(debut, fin)` → **yield** chaque port de `debut` à `fin` inclus.
- `ips_sous_reseau(prefixe, debut, fin)` → **yield** chaque IP "`prefixe.i`" pour `i` de `debut` à `fin`. Ex : `ips_sous_reseau("192.168.1", 1, 3)` produit "`192.168.1.1`", "`192.168.1.2`", "`192.168.1.3`".

Tests à faire passer :

```
print(list(plage_ports(20, 23)))
# -> [20, 21, 22, 23]

for ip in ips_sous_reseau("192.168.1", 1, 3):
    print(ip)
# -> 192.168.1.1
#     192.168.1.2
#     192.168.1.3

# Preuve que c'est un générateur (pas une liste) :
import sys
g = plage_ports(1, 1000000)
print(type(g)) # ->
print(sys.getsizeof(g) < 500) # -> True (quelques centaines d'octets, pas des Mo)

# Preuve de l'usage unique :
h = plage_ports(1, 3)
print(list(h)) # -> [1, 2, 3]
print(list(h)) # -> [] (épuisé)
```

Bonus : une expression génératrice qui, à partir de `plage_ports(1, 100)`, ne garde **que les ports bien connus** (< 1024, mais bornés à 100 ici) et pairs — en une ligne avec des parenthèses et un `if`.

✓ Critères de réussite

- `plage_ports` et `ips_sous_reseau` contiennent un `yield` (ce sont des générateurs).
- `list(plage_ports(20, 23))` renvoie `[20, 21, 22, 23]` (bornes incluses).
- `type(g)` est `<class 'generator'>` et sa taille mémoire est minuscule.
- Le 2e `list(h)` renvoie `[]` (tu as vu l'épuisement).

Si tu bloques — indices progressifs

Étape	Indice
<code>plage_ports</code>	Une boucle <code>for port in range(debut, fin + 1):</code> puis <code>yield port</code> . Le <code>+ 1</code> car <code>range</code> exclut la borne haute, et toi tu la veux incluse.
<code>ips_sous_reseau</code>	Même idée : <code>for i in range(debut, fin + 1):</code> puis <code>yield f"{prefixe}.{i}"</code> .
Vérifier le type	Dès qu'une fonction contient <code>yield</code> , l'appeler renvoie un objet générateur — tu n'as rien de plus à faire.

Où ranger : `mini-projets/11-generateurs/generateurs.py` .

Lien avec la suite : au Module réseau, tu boucleras sur `plage_ports(...)` pour tester chaque port sans jamais construire la liste des 65 535. C'est un vrai gain sur un scanner.

Quand c'est prêt : `git add .` → `commit` → `push` , puis « c'est poussé ».